САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики–процессов управления

Программа бакалавриата

«Большие данные и распределенная цифровая платформа»

ОТЧЕТ

По лабораторной работе № 2

По дисциплине «Алгоритмы и структуры данных»

На тему «Исследование задачи коммивояжера через алгоритм имитации отжига и муравьиный алгоритм с модификациями»

Студент гр. 24Б15-пу
Телицин А.Д.

Преподаватель
Дик А.Г.

Санкт-Петербург
2026 г.

Содержание

1 Цель работы **2**

2 Описание алгоритмов и постановка задачи **2**
2.1 Постановка задачи коммивояжера 2
2.2 Алгоритм имитации отжига 2
2.3 Модификация имитации отжига: больцмановское охлаждение 4
2.4 Муравьиный алгоритм 4
2.5 Модификация муравьиного алгоритма: «начальное расположение» . . . 5

3 Пошаговое описание выполнения алгоритмов **5**
3.1 Пошаговое описание: базовый алгоритм имитации отжига 5
3.2 Пошаговое описание: имитация отжига с больцмановским охлаждением 8
3.3 Пошаговое описание: базовый муравьиный алгоритм 9
3.4 Пошаговое описание: муравьиный алгоритм с модификацией «начальное расположение» 11

4 Рекомендации пользователю **12**

5 Рекомендации программисту **14**

6 Описание контрольного примера **15**

7 Исследование параметров и анализ результатов **16**
7.1 Методика исследования 16
7.2 Имитация отжига (на Berlin52) 16
7.3 Имитация отжига (на World666) 16
7.4 Муравьиный алгоритм (на Berlin52) 17
7.5 Муравьиный алгоритм (на World666) 18
7.6 Сводная таблица результатов 18
7.7 Сравнение алгоритмов 18

8 Выводы **19**

9 Листинг кода **21**

1 Цель работы

Целью работы является исследование и сравнительный анализ качества решения задачи коммивояжера с помощью:

1. базового алгоритма имитации отжига;
2. модифицированного алгоритма имитации отжига (больцмановское охлаждение);
3. базового муравьиного алгоритма;
4. модифицированного муравьиного алгоритма («начальное расположение»).

Основной акцент в сравнении делается на качестве получаемого маршрута (длине гамильтонова цикла), затем на времени выполнения и стабильности результатов при изменении параметров. Эксперименты проводятся на среднеразмерном графе Berlin52 и крупном графе World666.

2 Описание алгоритмов и постановка задачи

2.1 Постановка задачи коммивояжера

Пусть задан полный взвешенный граф $G = (V, E)$, где:

- $V = \{1, 2, \ldots, n\}$ – множество вершин (городов),
- E – множество ребер между всеми парами городов,
- $d_{ij} > 0$ – длина (вес) ребра между городами i и j, $d_{ii} = 0$.

Требуется найти гамильтонов цикл минимальной длины, то есть такую перестановку вершин $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$, при которой минимизируется функционал:

$$L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1} \to \min\,. \tag{1}$$

В рамках данной лабораторной работы выполняется сравнение четырех алгоритмических вариантов:

1. базовый алгоритм имитации отжига;
2. модификация имитации отжига с больцмановским расписанием охлаждения;
3. базовый муравьиный алгоритм;
4. модификация муравьиного алгоритма «начальное расположение».

2.2 Алгоритм имитации отжига

Алгоритм имитации отжига – это стохастический метод комбинаторной оптимизации, вдохновленный физическим процессом отжига металлов. В металлургии материал сначала нагревают до высокой температуры, а затем постепенно охлаждают. При высоких температурах атомы могут свободно переходить между состояниями, а при медленном охлаждении система имеет шанс перейти в состояние с минимальной энергией.

В задаче коммивояжера проводится следующая аналогия:

- состояние системы – конкретный маршрут коммивояжера;
- энергия состояния – длина маршрута;
- температура – параметр, управляющий вероятностью принятия ухудшающих переходов.

Основная идея метода состоит в том, что на ранних этапах (при высокой температуре) алгоритм имеет высокую степень свободы, чтобы не застревать в локальных минимумах, а по мере охлаждения становится более статичным и стабилизируется около лучших найденных решений.

Пусть:

- π – текущее решение,
- π' – соседнее решение,
- $L(\pi)$, $L(\pi')$ – длины маршрутов,
- $\Delta = L(\pi') - L(\pi)$,
- T – текущая температура.

Правило принятия решения:

$$\pi \leftarrow \begin{cases} \pi', & \Delta < 0, \\ \pi', & \Delta \geq 0 \text{ и } r < \exp\left(-\frac{\Delta}{T}\right), \\ \pi, & \text{иначе}, \end{cases} \tag{2}$$

где $r \sim U(0,1)$.

Если новое решение улучшает целевую функцию ($\Delta < 0$), оно принимается всегда. Если решение хуже ($\Delta \geq 0$), оно принимается с вероятностью Больцмана:

$$P(\text{accept}) = \exp\left(-\frac{\Delta}{T}\right). \tag{3}$$

Для базовой версии используется геометрическое охлаждение:

$$T_{k+1} = \alpha T_k, \quad 0 < \alpha < 1. \tag{4}$$

Для задачи коммивояжера критически важен способ генерации соседнего решения. В данной работе используется следующий подход: выбираются две позиции маршрута, а подмаршрут между ними разворачивается.

С практической точки зрения метод имитации отжига характеризуется следующими особенностями:

- хорошо подходит для сложных комбинаторных задач большой размерности;
- имеет небольшие вычислительные накладные расходы на одну итерацию;
- чувствителен к выбору расписания температуры и числу итераций;
- при слишком быстром охлаждении может рано зафиксироваться в локальном минимуме.

2.3 Модификация имитации отжига: больцмановское охлаждение

В модифицированном варианте используется другая функция температуры:

$$T(k) = \frac{T_0}{\ln(1+k)}, \tag{5}$$

где k – номер итерации ($k \geq 1$), T_0 – начальная температура.

Вероятность принятия ухудшающего шага сохраняется:

$$P(\text{accept}) = \exp\left(-\frac{\Delta}{T(k)}\right), \quad \Delta > 0. \tag{6}$$

По сравнению с геометрическим охлаждением данная схема убывает существенно медленнее, что повышает качество исследования пространства решений, но увеличивает время работы. На практике это может привести к значтиельному улучшению качества решения, но при этом требует больше времени на поиск.

2.4 Муравьиный алгоритм

Муравьиный алгоритм (Ant Colony Optimization, ACO) – это эвристический метод, основанный на идее коллективного поведения муравьев при поиске кратчайших путей к источнику пищи.

В природной системе муравьи оставляют на пути химический след (феромон). Чем чаще используется путь, тем больше на нем феромона, и тем вероятнее, что следующие муравьи выберут именно его. При этом за счет испарения феромона система не «застывает» и сохраняет способность к адаптации.

В задаче коммивояжера:

- каждый муравей строит полный маршрут по графу;
- качество маршрута оценивается его длиной;
- более короткие маршруты усиливаются большим количеством феромона.

Обозначим:

- τ_{ij} – уровень феромона на ребре (i, j);
- $\eta_{ij} = \frac{1}{d_{ij}}$ – эвристическая привлекательность ребра;
- α – степень влияния феромона;
- β – степень влияния эвристики;
- ρ – коэффициент испарения феромона;
- Q – коэффициент отложения феромона;
- M – число муравьев в колонии.

Вероятность перехода муравья из вершины i в вершину j:

$$P_{ij} = \frac{\tau_{ij}^{\alpha}\eta_{ij}^{\beta}}{\sum\limits_{k \in \mathcal{N}_i} \tau_{ik}^{\alpha}\eta_{ik}^{\beta}}, \tag{7}$$

где $\mathcal{N}_i$ – множество еще не посещенных вершин.

После завершения маршрутов всеми муравьями выполняется глобальное обновление феромона. Сначала моделируется испарение:

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij}. \tag{8}$$

Затем добавляется вклад всех муравьев:

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{m=1}^{M} \Delta\tau_{ij}^{(m)}, \tag{9}$$

где M – число муравьев, а вклад m-го муравья задается:

$$\Delta\tau_{ij}^{(m)} = \begin{cases} \frac{Q}{L_m}, & \text{если ребро } (i,j) \text{ входит в маршрут муравья } m, \\ 0, & \text{иначе,} \end{cases} \tag{10}$$

L_m – длина маршрута муравья m.

Итоговая формула обновления:

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \sum_{m=1}^{M} \Delta\tau_{ij}^{(m)}. \tag{11}$$

Таким образом, алгоритм сочетает два механизма:

- Сохранение предыдущего опыта решений через отложение феромона, что способствует усилению хороших маршрутов.
- Способность к поиску новых решений за счет вероятностного выбора и испарения феромона, что предотвращает преждевременную сходимость.

2.5 Модификация муравьиного алгоритма: «начальное расположение»

В базовом варианте стартовая вершина каждого муравья выбирается случайно. В модификации «начальное расположение» первая итерация организуется так, чтобы муравьи стартовали из разных вершин более равномерно:

$$s_m = m \bmod n, \tag{12}$$

где s_m – стартовая вершина муравья m, n – число вершин.

Далее алгоритм работает как базовый ACO. Цель модификации – улучшить начальное покрытие пространства решений и уменьшить вероятность преждевременной концентрации феромона в ограниченной области графа.

3 Пошаговое описание выполнения алгоритмов

3.1 Пошаговое описание: базовый алгоритм имитации отжига

1. **Инициализация.** Формируется начальный маршрут $\pi^{(0)}$, задаются начальная температура T_0, коэффициент охлаждения α, максимальное число итераций $K_{\max}$. Вычисляется начальное значение целевой функции $L(\pi^{(0)})$.
2. **Инициализация лучшего решения.** Текущее решение принимается как лучшее: $\pi^{best} \leftarrow \pi^{(0)}$, $L^{best} \leftarrow L(\pi^{(0)})$.

3. **Генерация соседнего решения.** На итерации k из текущего маршрута $\pi^{(k)}$ строится соседний маршрут π' оператором 2-opt.

4. **Оценка соседнего решения.** Вычисляется $\Delta = L(\pi') - L(\pi^{(k)})$.

5. **Правило перехода.** Если $\Delta < 0$, переход выполняется всегда. Если $\Delta \geq 0$, переход выполняется с вероятностью $P = \exp\left(-\frac{\Delta}{T_k}\right)$.

6. **Обновление лучшего решения.** Если новое текущее решение лучше глобально лучшего, то: $\pi^{best} \leftarrow \pi^{(k+1)}$, $L^{best} \leftarrow L(\pi^{(k+1)})$.

7. **Охлаждение температуры.** Температура обновляется по формуле: $T_{k+1} = \alpha T_k$.

8. **Проверка критерия остановки.** Если достигнуто $K_{\max}$ или $T_k < T_{\min}$, выполняется остановка. Иначе переход к шагу 3.

9. **Завершение алгоритма.** Возвращается лучшее найденное решение π^{best} и его длина L^{best}.

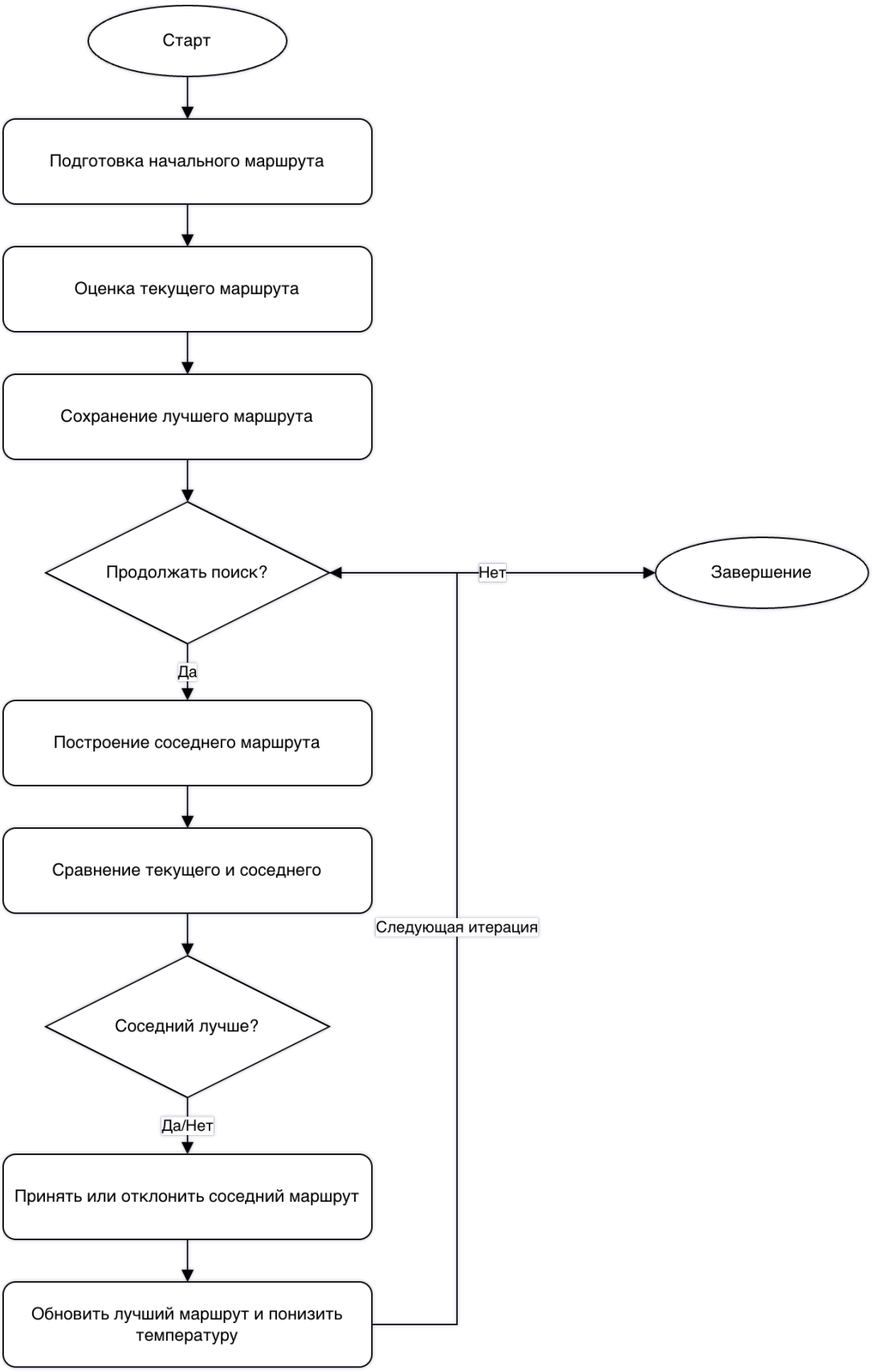

Рис. 1: Блок-схема базового алгоритма имитации отжига.

3.2 Пошаговое описание: имитация отжига с больцмановским охлаждением

1. **Инициализация.** Формируется начальный маршрут $\pi^{(0)}$, задаются T_0 и $K_{\max}$.

2. **Инициализация лучшего решения.** Устанавливается: $\pi^{best} \leftarrow \pi^{(0)}$, $L^{best} \leftarrow L(\pi^{(0)})$.

3. **Вычисление температуры по номеру итерации.** Для текущей итерации k:
$$T(k) = \frac{T_0}{\ln(1+k)}.$$

4. **Генерация соседнего решения.** Строится π' из $\pi^{(k)}$ оператором 2-opt.

5. **Оценка перехода и принятие решения.** Вычисляется $\Delta = L(\pi') - L(\pi^{(k)})$. Применяется то же правило принятия, что и в базовом SA.

6. **Обновление лучшего найденного решения.** При улучшении обновляются π^{best} и L^{best}.

7. **Проверка критерия остановки.** Если $k \geq K_{\max}$ или $T(k) < T_{\min}$, переход к завершению. Иначе переход к шагу 3 следующей итерации.

8. **Завершение алгоритма.** Возвращается π^{best} и L^{best}.

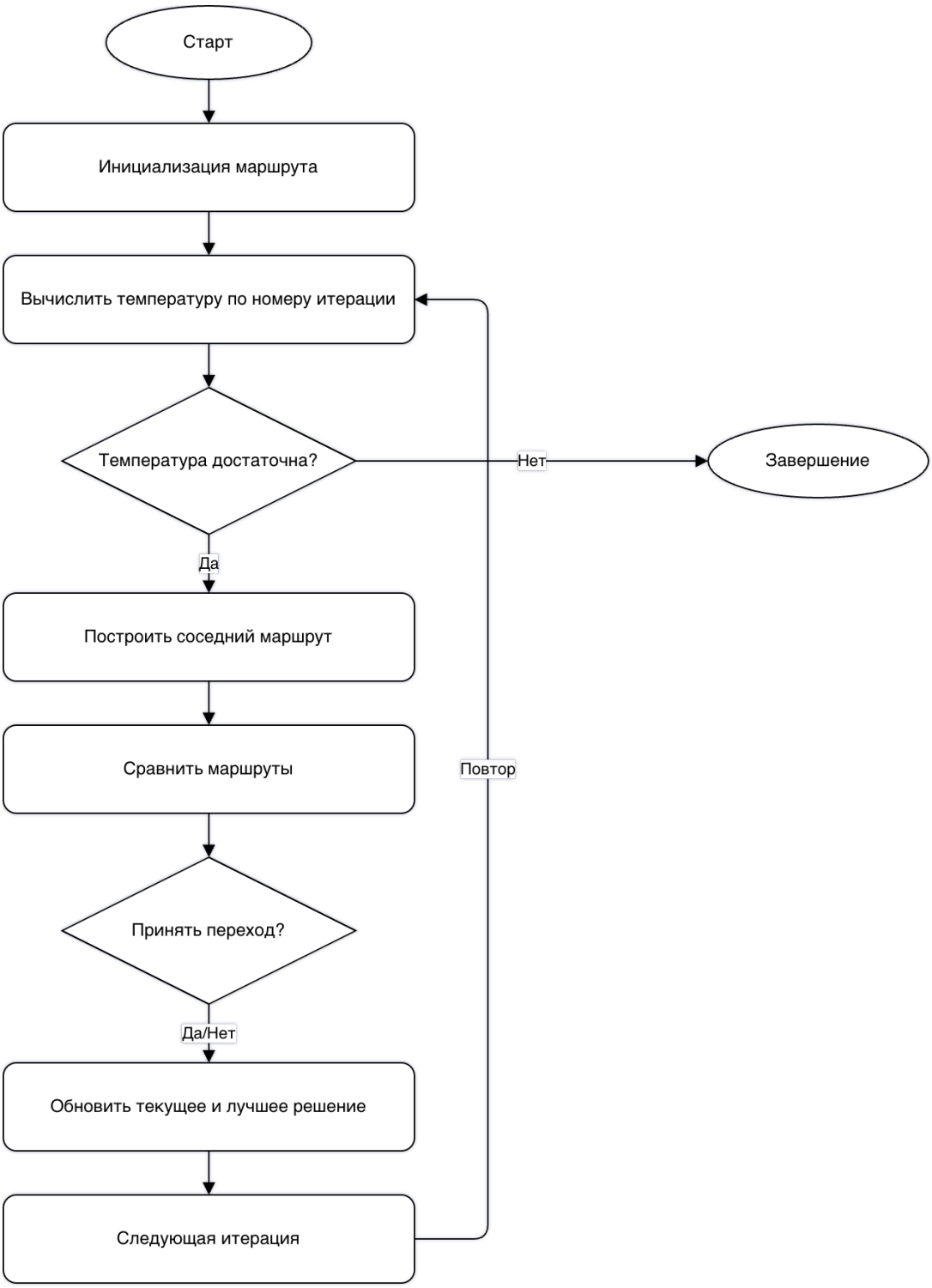

Рис. 2: Блок-схема алгоритма имитации отжига с больцмановским охлаждением.

3.3 Пошаговое описание: базовый муравьиный алгоритм

1. **Инициализация параметров.** Задаются M (число муравьев), $I_{\max}$ (число итераций), α, β, ρ, Q.

2. **Инициализация феромона.** Для всех ребер (i, j) задается начальный уровень феромона $\tau_{ij}^{(0)}$. Вычисляется эвристика $\eta_{ij} = 1/d_{ij}$.

3. **Начало итерации колонии.** Для итерации t каждый муравей начинает по-

строение маршрута из стартовой вершины (в базовой версии старт выбирается случайно).

4. **Построение маршрутов муравьями.** Каждый муравей пошагово выбирает следующую непосещенную вершину по вероятностному правилу P_{ij} до построения полного гамильтонова цикла.

5. **Оценка маршрутов.** Для каждого муравья вычисляется длина маршрута L_m. Обновляется глобально лучший маршрут при необходимости.

6. **Испарение феромона.** Выполняется:

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij}.$$

7. **Отложение феромона.** Для всех муравьев добавляется вклад:

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{m=1}^{M} \Delta\tau_{ij}^{(m)}.$$

8. **Проверка критерия остановки.** Если $t \geq I_{\max}$, переход к завершению. Иначе переход к шагу 3.

9. **Завершение алгоритма.** Возвращается лучший найденный маршрут и его длина.

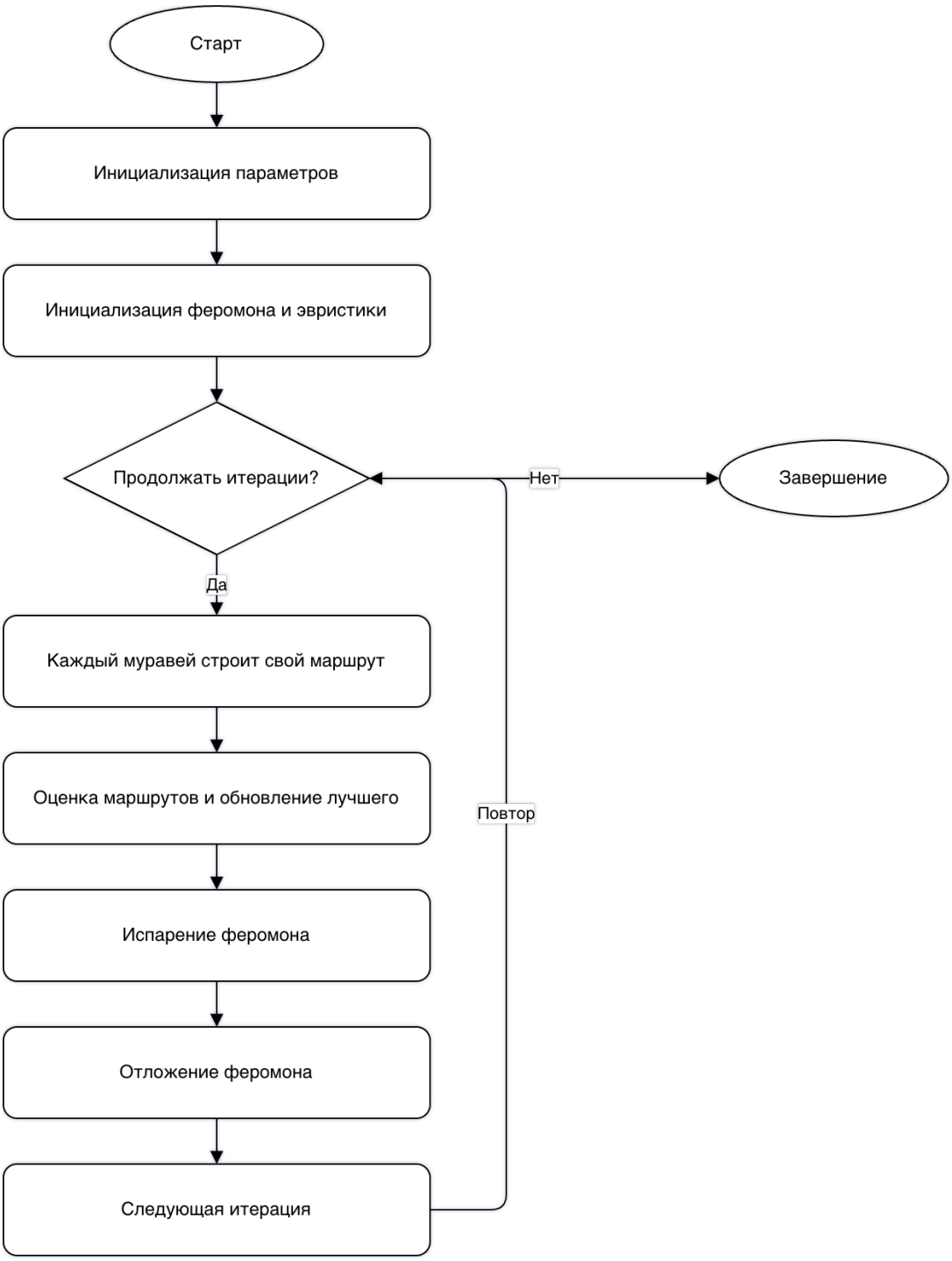

Рис. 3: Блок-схема базового муравьиного алгоритма.

3.4 Пошаговое описание: муравьиный алгоритм с модификацией «начальное расположение»

1. **Инициализация параметров и феромона.** Выполняется аналогично базовому ACO.

2. **Специальная инициализация стартовых вершин (первая итерация).** На первой итерации старт муравья m задается по правилу:

$$s_m = m \bmod n.$$

Это обеспечивает равномерное покрытие графа в начале работы алгоритма.

3. **Построение маршрутов и их оценка.** Выполняется как в базовом ACO с обновлением лучшего решения.

4. **Обновление феромона.** Испарение и отложение выполняются как в базовом ACO.

5. **Последующие итерации.** Начиная со второй итерации алгоритм работает как базовый ACO (со случайным или стандартным стартом, в зависимости от реализации).

6. **Проверка критерия остановки.** Если достигнуто $I_{\max}$, переход к завершению. Иначе повторение цикла итераций.

7. **Завершение алгоритма.** Возвращается лучший найденный маршрут и его длина.

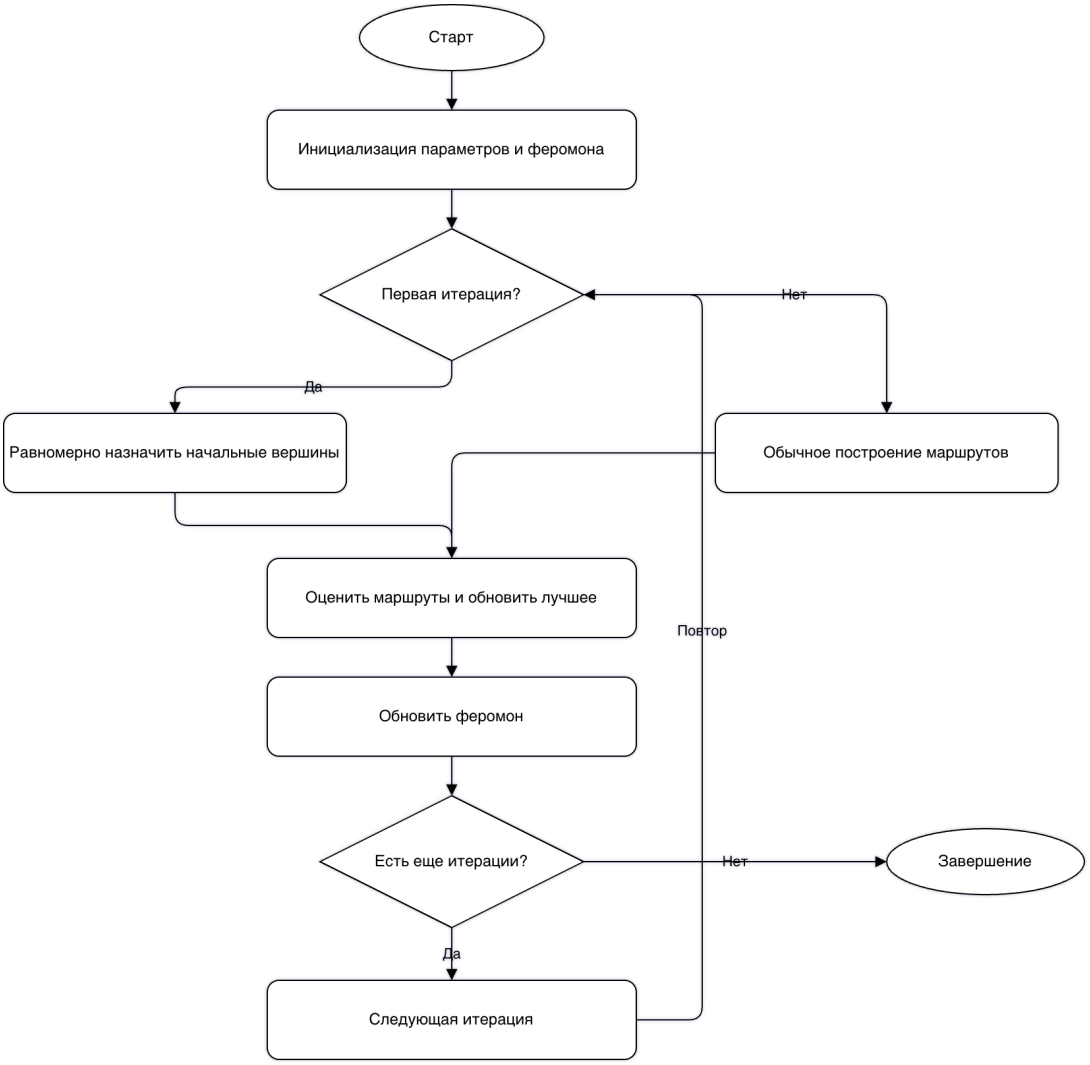

Рис. 4: Блок-схема муравьиного алгоритма с модификацией «начальное расположение».

4 Рекомендации пользователю

Взаимодействие с программой осуществляется через графический интерфейс.

1. **Запуск программы.** Активируйте виртуальное окружение и выполните команду:

```
python3 main.py
```

2. Откроется окно, где можно выбрать граф, алгоритм и параметры.

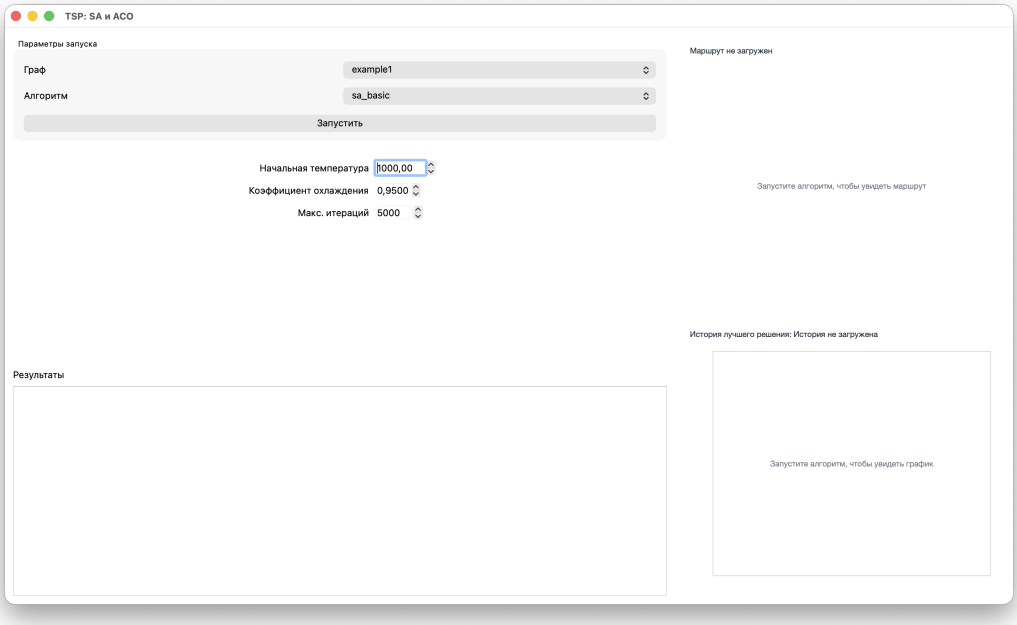

Рис. 5: Интерфейс программы.

3. **Выбор алгоритма.** Производится через выпадающий список в интерфейсе. На выбор представлены четыре варианта: базовый и больцмановский отжиг, базовый и с начальным расположением муравьиный алгоритм.

4. **Выбор графа.** Производится через выпадающий список. Доступные графы: *small*, *berlin52* и *world666*.

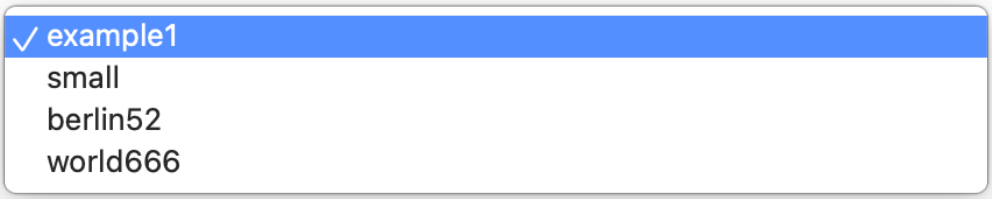

Рис. 6: Интерфейс программы.

5. **Настройка параметров.** Программа позволяет изменять ключевые параметры алгоритмов, такие как начальная температура, коэффициент охлаждения для отжига, а также α, β, ρ и Q для муравьиного алгоритма. Рекомендуется начинать с базовых значений и постепенно их изменять.

6. **Результаты.** После запуска следует ориентироваться не только на длину найденного маршрута, но и на график сходимости. Если кривая быстро выходит на плато, это означает, что выбранные параметры слишком быстро ограничивают поиск.

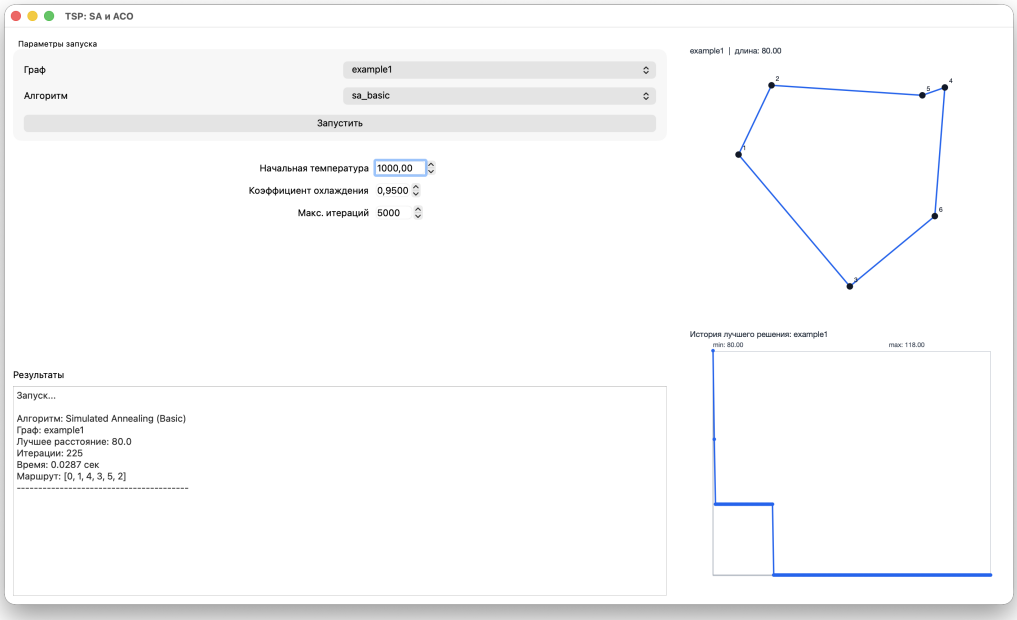

Рис. 7: Интерфейс программы.

5 Рекомендации программисту

Системные требования:

- операционная система macOS, Linux или Windows;
- Python 3.14 или совместимая версия Python 3;
- установленный пакет PyQt6 для запуска графического интерфейса;
- установленный NumPy для вычислений и построения раскладки графа.

Структура проекта:

- `src/graph_utils.py` – загрузка графов и вычисление длины маршрута;
- `src/annealing.py` – реализация базового и больцмановского отжига;
- `src/aco.py` – реализация базового ACO и модификации initial placement;
- `src/ui.py` – графический интерфейс, запуск алгоритмов и визуализация;
- `main.py` – точка входа в приложение.

Инструкция по запуску:

1. Активировать виртуальное окружение.
2. Установить зависимости из `requirements.txt`.
3. Запустить `python3 main.py`.
4. Выбрать граф, алгоритм и параметры в окне программы.

6 Описание контрольного примера

В качестве контрольного примера использован малый граф *small* из шести вершин. Этот пример удобен тем, что он не требует значительных вычислительных ресурсов, а оптимальный маршрут известен заранее (длина 10). Это позволяет проверить корректность реализации алгоритмов и убедиться,

В ходе проверки были получены следующие результаты:

Таблица 1: Результаты контрольного прогона на графе *small*.

Алгоритм	Длина маршрута	Итерации	Время, сек
Базовая имитация отжига	10	225	0.0023
Имитация отжига с больцмановским охлаждением	10	1000	0.0093
Базовый муравьиный алгоритм	10	50	0.0419
Муравьиный алгоритм + начальное расположение	10	50	0.0669

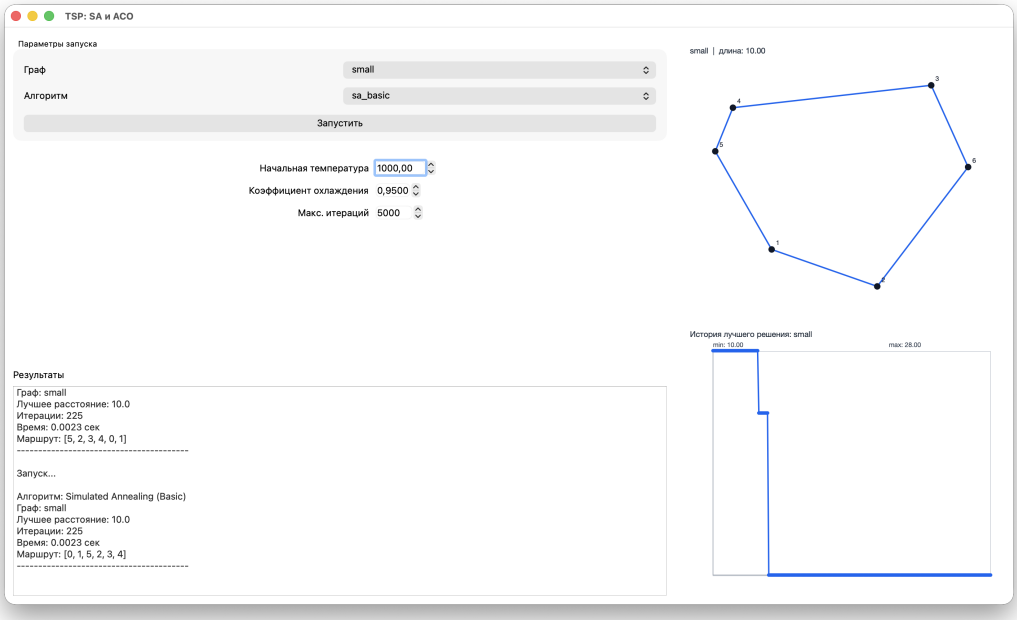

Рис. 8: Интерфейс программы.

Полученные значения подтверждают корректность реализации: все четыре варианта алгоритмов находят оптимальный маршрут длины 10 на контрольном графе. Визуализация в графическом интерфейсе показывает найденный маршрут на двумерной раскладке графа. График сходимости найденного решения демонстрирует, что алгоритмы повышают качество решения по мере итераций.

7 Исследование параметров и анализ результатов

7.1 Методика исследования

Для оценки влияния параметров на качество решения были выполнены серии экспериментов для графов Berlin52 и World666. В каждой серии изменялся один параметр, а остальные оставались фиксированными. В качестве метрик, на основе которых проводилось сравнение алгоритмов были выбраны следующие (в порядке убывания значимости):

- **Длина найденного маршрута** – основная метрика качества решения.
- **Время работы** – важный показатель эффективности алгоритма.
- **Устойчивость результатов** – оценка вариативности решения при повторных запусках с разными начальными условиями.

Для Berlin52 эксперименты выполнялись с небольшим числом повторов, а для World666 число повторов было сокращено, в целях снижения общего времени вычислений. Поэтому при интерпретации результатов основное внимание уделяется не абсолютной точности статистики, а общим тенденциям поведения алгоритмов.

7.2 Имитация отжига (на Berlin52)

В базовом варианте имитации отжига наибольшее влияние на качество решения оказал выбор начальной температуры T_0 и коэффициента охлаждения α. На Berlin52 оптимальные результаты были получены при $T_0 = 800$ и $\alpha = 0.9989$, что обеспечивало достаточно медленное охлаждение. При более быстром охлаждении ($\alpha < 0.95$) в сочетании с низкой начальной температурой алгоритм часто застревал в локальных минимумах. Полученное расстояние составило 1980, но при этом данный результат не был стабильным и при повторных запусках мог значительно ухудшаться. Время выполнения при данных параметрах составило 0.1893 секунды, что является приемлемым для данной задачи.

В варианте с больцмановским охлаждением оптимальные результаты были достигнуты при $T_0 = 40$ и $K_{\max} = 10000$. Этот вариант показал немного более стабильные результаты, с меньшей вариативностью между запусками, лучшее найденное расстояние составило 1941, а время выполнения осталось приблизительно тем же, что и в варианте без модификации, 0.1832 секунды. Особенностью больцмановского охлаждения является более медленное снижение температуры, что позволяет алгоритму более тщательно исследовать пространство решений, но при этом требует другого подхода к выбору параметров, так например, $T_0 = 40$ является низким относительно базового отжига, но при этом обеспечивает более эффективный поиск при медленном охлаждении. Более высокие значения T_0 в данном случае приводили к избыточной случайности и ухудшению результатов.

7.3 Имитация отжига (на World666)

Базовый алгоритм имитации отжига на World666 показал более высокую чувствительность к параметрам. Наилучшие результаты были получены при $T_0 = 800$ и $\alpha = 0.9999$, что обеспечивало достаточно медленное охлаждение для такого большого графа. Полученное расстояние составило 584576, а время выполнения – 16.0894 секунды. Число ограничения итераций было выбрано таким образом, чтобы алгоритм мог стабилизироваться около лучших решений, что особенно важно при медленном

охлаждении. При более быстром охлаждении алгоритм демонстрировал более высокую вариативность и хуже качество решения, а при более низких T_0 он часто застревал в локальных минимумах.

Алгоритм в варианте с больцмановским охлаждением показал наиболее качественные решения при $T_0 = 60$ и $K_{\max} = 100000$. Полученное расстояние составило 609483. Время выполнения составило 15.8380 секунды. Выбор таким параметров был обусловлен размером графа и необходимостью обеспечить достаточно медленное охлаждение для эффективного поиска. При более высоких T_0 алгоритм демонстрировал более высокую вариативность и хуже качество решения, а при более низких T_0 он часто застревал в локальных минимумах. Ограничение итераций было выбрано таким образом, чтобы алгоритм мог стабилизироваться около лучших решений, что особенно важно при медленном охлаждении.

Таким образом, для имитации отжига можно сделать следующие выводы:

- оба алгоритма позволяют достичь хороших результатов при правильной настройке параметров, однако
- наибольшее влияние на характер работы алгоритма оказывает выбор и настройка функции температуры;
- слишком быстрое охлаждение ухудшает поиск на обоих графах;
- больцмановская схема требует больше итераций для достижения хорошего результата, но при этом может значительно улучшить качество решения;
- больцмановская схема требует иного подхода к выбору начальной температуры, которая должна быть достаточно низкой для эффективного поиска при медленном охлаждении;
- важным является выбор ограничений итераций таким образом, чтобы алгоритм работал в том числе и при более низких температурах, что позволяет ему стабилизироваться около лучших решений.

7.4 Муравьиный алгоритм (на Berlin52)

Для муравьиного алгоритма наиболее значимым оказался баланс между влиянием феромона и эвристики. На обеих задачах улучшение результатов наблюдалось при увеличении α и β по сравнению с базовыми значениями. Это означает, что алгоритму полезно сильнее учитывать как уже найденные хорошие ребра, так и текущую привлекательность коротких переходов.

На Berlin52 наилучшие результаты среди проверенных серий были получены при количестве муравьев $M = 30$, количестве итераций $I_{\max} = 50$, $\alpha = 1.0$ и $\beta = 2.0$. При этом уменьшение или увеличение ρ вокруг $0.3 - 0.6$ приводило к ухудшению результата, то есть слишком слабое испарение приводило к ранней фиксации феромона, а слишком сильное – к потере накопленной информации. Полученный результат составил 1928, а время выполнения – 1.3808 секунды.

С использованием модификации «начальное расположение» на Berlin52 были получены лучшие результаты при $M = 30$, $I_{\max} = 50$, $\alpha = 1.5$, $\beta = 2.0$ и $Q = 50$. Этот вариант показал улучшение качества решения, что может быть связано с более равномерным начальным покрытием графа, что позволяет алгоритму эффективнее исследовать пространство решений. Полученный результат составил 1942, а время выполнения – 1.3739 секунды.

7.5 Муравьиный алгоритм (на World666)

На World666 испытания были осложнены большим размером графа, что ограничивало число повторов. Тем не менее, были подобраны следующие параметры: $M = 30$, $I_{\max} = 50$, $\alpha = 1.5$, $\beta = 2.0$ и $\rho = 0.4$, что является той же самой конфигурацией, что и для Berlin52. Полученное расстояние составило 397880, а время выполнения – 131.4481 секунды.

Для модификации «начальное расположение» характерна более равномерная начальная разведка графа. На Berlin52 это дало небольшое преимущество на некоторых параметрах, а на World666 различия между базовым и модифицированным вариантом были менее однозначными. Тем не менее, модификация полезна как способ уменьшить зависимость результата от случайного старта. Полученный результат для World666 составил 401051, а время выполнения – 171.5460 секунды при использовании параметров $M = 30$, $I_{\max} = 50$, $\alpha = 1.5$, $\beta = 3.0$, $\rho = 0.4$ и $Q = 100$.

7.6 Сводная таблица результатов

Таблица 2: Ключевые результаты исследования параметров на графе Berlin52.

Алгоритм	Наиболее удачные параметры в серии испытаний	Длина маршрута
Имитация отжига (базовый)	$\alpha = 0.9989$; $T_0 = 800$	1980
Имитация отжига (Больцман)	$T_0 = 40$; $K_{\max} = 10000$	1941
Муравьиный алгоритм (базовый)	$\alpha = 1.0$; $\beta = 2.0$; $\rho = 0.4$	1928
Муравьиный алгоритм (начальное расположение)	$\alpha = 1.5$; $\beta = 2.0$; $Q = 50$	1942

Таблица 3: Ключевые результаты исследования параметров на графе World666.

Алгоритм	Наиболее удачные параметры в серии испытаний	Длина маршрута
Имитация отжига (базовый)	$\alpha = 0.9999$; $T_0 = 800$	584576
Имитация отжига (Больцман)	$T_0 = 60$; $K_{\max} = 100000$	609483
Муравьиный алгоритм (базовый)	$\alpha = 1.5$; $\beta = 2.0$; $\rho = 0.4$	397880
Муравьиный алгоритм (начальное расположение)	$\alpha = 1.5$; $\beta = 3.0$; $\rho = 0.4$; $Q = 100$	401051

7.7 Сравнение алгоритмов

С точки зрения **качества** на графе Berlin52 оба алгоритма обеспечили поиск решения, близкого к оптимальному (1928 против 1941-1942), при этом муравьиный

алгоритм показал чуть лучшее качество при больших временных затратах.

На графе World666 разница в качестве стала более заметной: муравьиный алгоритм обеспечил значительно более короткий маршрут (397880 против 584576-609483), что свидетельствует о его лучшей способности находить хорошие решения на больших и сложных графах. При этом стоит учитывать, что время выполнения муравьиного алгоритма было существенно больше, что может быть критичным в условиях ограниченных ресурсов.

С точки зрения **скорости** имитация отжига работает существенно быстрее, чем муравьиный алгоритм, поскольку на каждой итерации обрабатывает только одно соседнее решение, тогда как ACO строит множество маршрутов сразу для всей колонии.

С точки зрения **стабильности** муравьиный алгоритм показал более устойчивые результаты при повторных запусках, особенно на большом графе, что может быть связано с его механизмом коллективного обучения через феромоны. Еще сильнее увеличивает стабильность модификация «начальное расположение», которая обеспечивает более равномерное начальное покрытие графа.

Имитация отжига, особенно в базовом варианте, была более чувствительна к начальным условиям и параметрам, что приводило к большей вариативности результатов. Модификация с больцмановским охлаждением улучшила стабильность, но при этом потребовала более тщательного подбора параметров и увеличила время выполнения.

В результате можно сделать итоговый вывод: допустимо использование обоих алгоритмов, однако при приоритете качества решения для задач средней и большой размерности предпочтителен муравьиный алгоритм с тщательным подбором параметров. При приоритете времени вычислений может использоваться имитация отжига, но это обычно приводит к ухудшению качества решения на сложных графах.

8 Выводы

В ходе лабораторной работы была исследована задача коммивояжера с применением четырех вариантов алгоритмов: базовой имитации отжига, имитации отжига с больцмановским охлаждением, базового муравьиного алгоритма и муравьиного алгоритма с модификацией «начальное расположение».

По результатам экспериментов на графах Berlin52 и World666 получены следующие основные выводы.

1. **По критерию качества решения** на обеих задачах муравьиный алгоритм показал лучшие результаты, чем имитация отжига. Разница особенно заметна на графе World666, где ACO дает существенно более короткие маршруты.

2. **По критерию скорости** имитация отжига работает быстрее, поскольку на каждой итерации оценивается одно соседнее решение, тогда как муравьиный алгоритм строит набор маршрутов всей колонией.

3. **По критерию стабильности** и управляемости результата ключевую роль играют параметры алгоритмов. Для имитации отжига наиболее критичен закон охлаждения (в первую очередь коэффициент охлаждения в базовой версии). Для ACO наиболее значимы параметры α и β, задающие баланс между феромоном и эвристической информацией.

4. Больцмановская модификация отжига обеспечивает более глубокий поиск за счет медленного снижения температуры и на сложных графах может замет-

но улучшать качество по сравнению с базовой версией, но ценой увеличения времени.

5. Модификация муравьиного алгоритма «начальное расположение» улучшает начальное покрытие графа и в ряде режимов дает преимущество по качеству решения, особенно на средних размерах задачи.

Итоговый практический вывод: при приоритете **качества решения** для задачи коммивояжера на графах средней и большой размерности предпочтителен муравьиный подход с подбором параметров. При приоритете **времени вычислений** может использоваться имитация отжига, однако это обычно приводит к ухудшению качества маршрута на сложных графах.

Полученные результаты корректны для конкретных реализаций алгоритмов и выбранных графов, и могут отличаться при других настройках или на других задачах. Тем не менее, общие тенденции в поведении алгоритмов сохраняются, что позволяет сделать обоснованные рекомендации по их применению в зависимости от требований к качеству решения и доступного времени.

9 Листинг кода

В данном разделе приведены исходные коды основных модулей программы.

Листинг 1. Исходный код файла `main.py`

```
from src.ui import run_app


if __name__ == '__main__':
    run_app()
```

Листинг 2. Исходный код файла `src/graph_utils.py`

```
import numpy as np
from typing import Tuple, Dict, List
from pathlib import Path


class Graph:
    """     ."""

    def __init__(self, distance_matrix: np.ndarray):
        """
        Args:
            distance_matrix:    (n, n)
        """
        self.distance_matrix = distance_matrix
        self.n_cities = len(distance_matrix)
        self._visualization_layout = None

    @staticmethod
    def create_small_graph() -> 'Graph':
        """
             6 .
           .
        """
        #    6  ()
        distances = np.array([
            [0, 10, 15, 20, 25, 30],
            [10, 0, 35, 25, 15, 20],
            [15, 35, 0, 30, 20, 10],
            [20, 25, 30, 0, 14, 16],
            [25, 15, 20, 14, 0, 18],
            [30, 20, 10, 16, 18, 0]
        ])
        return Graph(distances)

    @staticmethod
    def load_from_stp(filepath: str) -> 'Graph':
        """
           STP .
```

```
        Args:
            filepath:   STP

        Returns:
            Graph
        """
        filepath = Path(filepath)

        with open(filepath, 'r') as f:
            lines = f.readlines()

        #
        n_cities = None
        in_graph_section = False
        edges = []

        for line in lines:
            line = line.strip()

            if line.startswith('Section Graph'):
                in_graph_section = True
                continue

            if not in_graph_section:
                continue

            if line.startswith('Nodes'):
                n_cities = int(line.split()[1])
                continue

            if line.startswith('E '):
                parts = line.split()
                if len(parts) >= 4:
                    node1 = int(parts[1]) - 1  #   0
                    node2 = int(parts[2]) - 1
                    distance = float(parts[3])
                    edges.append((node1, node2, distance))

        #
        distance_matrix = np.zeros((n_cities, n_cities))
        for node1, node2, distance in edges:
            distance_matrix[node1][node2] = distance
            distance_matrix[node2][node1] = distance  #

        return Graph(distance_matrix)

    def get_distance(self, city1: int, city2: int) -> float:
        """    ."""
        return self.distance_matrix[city1][city2]

    def calculate_tour_length(self, tour: List[int]) -> float:
        """
           .
```

```
        Args:
            tour:

        Returns:
               (    )
        """
        length = 0
        for i in range(len(tour)):
            current = tour[i]
            next_city = tour[(i + 1) % len(tour)]
            length += self.get_distance(current, next_city)
        return length

    def get_visualization_layout(self) -> np.ndarray:
        """
                MDS.

        Returns:
               (n, 2),   .
        """
        if self._visualization_layout is not None:
            return self._visualization_layout.copy()

        distances = np.asarray(self.distance_matrix, dtype=float)
        n = distances.shape[0]
        if n == 0:
            self._visualization_layout = np.empty((0, 2), dtype=float)
            return self._visualization_layout.copy()

        squared = distances ** 2
        centering = np.eye(n) - np.ones((n, n)) / n
        gram = -0.5 * centering @ squared @ centering

        eigenvalues, eigenvectors = np.linalg.eigh(gram)
        order = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[order]
        eigenvectors = eigenvectors[:, order]

        top_values = np.clip(eigenvalues[:2], a_min=0.0, a_max=None)
        coords = eigenvectors[:, :2] * np.sqrt(top_values)

        if coords.shape[1] < 2:
            coords = np.pad(coords, ((0, 0), (0, 2 - coords.shape[1])), mode="constant")

        coords = coords - coords.mean(axis=0, keepdims=True)
        spread = np.max(np.linalg.norm(coords, axis=1))
        if spread > 0:
            coords = coords / spread

        self._visualization_layout = coords
        return coords.copy()
```

Листинг 3. Исходный код файла `src/annealing.py`

```
import numpy as np
from typing import List, Tuple, Dict
import time
import math


class SimulatedAnnealing:
    """
          .

        :
    P(delta) = exp(-delta / T),  delta = E_new - E_old, T -
    """

    def __init__(self, graph, initial_temp: float = 1000,
                 cooling_rate: float = 0.95, max_iterations: int = 10000):
        """
        Args:
            graph: Graph
            initial_temp:
            cooling_rate:   (0 < rate < 1)
            max_iterations:
        """
        self.graph = graph
        self.n_cities = graph.n_cities
        self.initial_temp = initial_temp
        self.cooling_rate = cooling_rate
        self.max_iterations = max_iterations

        #
        self.best_tour = None
        self.best_distance = float('inf')
        self.history = []

    def _random_tour(self) -> List[int]:
        """   ."""
        tour = list(range(self.n_cities))
        np.random.shuffle(tour)
        return tour

    def _get_neighbor(self, tour: List[int]) -> List[int]:
        """
           2-opt .
        2-opt:         .
        """
        neighbor = tour.copy()
        i, j = sorted(np.random.choice(self.n_cities, 2, replace=False))
        neighbor[i:j+1] = reversed(neighbor[i:j+1])
        return neighbor

    def solve(self) -> Tuple[List[int], float, Dict]:
        """
```

```
        .

    Returns:
        (best_tour, best_distance, metrics)
    """
    start_time = time.time()

    #
    current_tour = self._random_tour()
    current_distance = self.graph.calculate_tour_length(current_tour)

    self.best_tour = current_tour.copy()
    self.best_distance = current_distance

    temperature = self.initial_temp
    iterations = 0

    while temperature > 0.01 and iterations < self.max_iterations:
        #
        neighbor_tour = self._get_neighbor(current_tour)
        neighbor_distance = self.graph.calculate_tour_length(neighbor_tour)

        #
        delta = neighbor_distance - current_distance

        # ,
        if delta < 0 or np.random.rand() < math.exp(-delta / temperature):
            current_tour = neighbor_tour
            current_distance = neighbor_distance

        #
        if current_distance < self.best_distance:
            self.best_tour = current_tour.copy()
            self.best_distance = current_distance

        #
        self.history.append(self.best_distance)

        #
        temperature *= self.cooling_rate
        iterations += 1

    elapsed_time = time.time() - start_time

    metrics = {
        'algorithm': 'Simulated Annealing (Basic)',
        'best_distance': self.best_distance,
        'iterations': iterations,
        'time': elapsed_time,
        'final_temperature': temperature,
        'function_evaluations': iterations,
    }
```

```
        return self.best_tour, self.best_distance, metrics


class BoltzmannAnnealing(SimulatedAnnealing):
    """
       -  .

    :     .
     : T(k) = T_0 / ln(1 + k),  k -
            .
    """

    def __init__(self, graph, initial_temp: float = 1000, max_iterations: int = 10000):
        """
        Args:
            graph: Graph
            initial_temp:
            max_iterations:
        """
        super().__init__(graph, initial_temp, cooling_rate=1.0, max_iterations=max_iterations
        self.initial_temp = initial_temp

    def solve(self) -> Tuple[List[int], float, Dict]:
        """
            .

         :     T(k) = T_0 / ln(1 + k)

        Returns:
            (best_tour, best_distance, metrics)
        """
        start_time = time.time()

        #
        current_tour = self._random_tour()
        current_distance = self.graph.calculate_tour_length(current_tour)

        self.best_tour = current_tour.copy()
        self.best_distance = current_distance

        iterations = 0

        while iterations < self.max_iterations:
            #  : T(k) = T_0 / ln(1 + k)
            temperature = self.initial_temp / math.log(1 + iterations + 1)

            if temperature < 0.01:
                break

            #
            neighbor_tour = self._get_neighbor(current_tour)
            neighbor_distance = self.graph.calculate_tour_length(neighbor_tour)
```

```
            #
            delta = neighbor_distance - current_distance

            # ,
            if delta < 0 or np.random.rand() < math.exp(-delta / temperature):
                current_tour = neighbor_tour
                current_distance = neighbor_distance

            #
            if current_distance < self.best_distance:
                self.best_tour = current_tour.copy()
                self.best_distance = current_distance

            #
            self.history.append(self.best_distance)

            iterations += 1

        elapsed_time = time.time() - start_time

        metrics = {
            'algorithm': 'Simulated Annealing (Boltzmann)',
            'best_distance': self.best_distance,
            'iterations': iterations,
            'time': elapsed_time,
            'final_temperature': temperature,
            'function_evaluations': iterations,
        }

        return self.best_tour, self.best_distance, metrics
```

Листинг 4. Исходный код файла `src/aco.py`

```
import numpy as np
from typing import List, Tuple, Dict
import time


class AntColonyOptimization:
    """
         .

    :
    -    j   i: P_ij = _ij^ * _ij^ / (_ik^ * _ik^)
    -  : _ij = (1 - ) * _ij + (_ij)
    -  : _ij = Q / L_k (  k   L_k)

    :
    _ij -     (i,j)
    _ij = 1 / d_ij -  ()
     -  ,  -
     -
    Q -  ,
```

```
    """

    def __init__(self, graph, n_ants: int = 30, n_iterations: int = 100,
                 alpha: float = 1.0, beta: float = 2.0,
                 rho: float = 0.1, Q: float = 100):
        """
        Args:
            graph: Graph
            n_ants:
            n_iterations:
            alpha:   (  )
            beta:   (  )
            rho:    (0 < rho < 1)
            Q:  ,
        """
        self.graph = graph
        self.n_cities = graph.n_cities
        self.n_ants = n_ants
        self.n_iterations = n_iterations
        self.alpha = alpha
        self.beta = beta
        self.rho = rho
        self.Q = Q

        #
        initial_pheromone = 1.0 / (self.n_cities * 10)
        self.pheromone = np.full((self.n_cities, self.n_cities), initial_pheromone)
        np.fill_diagonal(self.pheromone, 0)  #

        #
        self.best_tour = None
        self.best_distance = float('inf')
        self.history = []

    def _calculate_heuristic(self) -> np.ndarray:
        """
           _ij = 1 / d_ij
        """
        heuristic = np.zeros((self.n_cities, self.n_cities))
        for i in range(self.n_cities):
            for j in range(self.n_cities):
                if i != j:
                    distance = self.graph.get_distance(i, j)
                    if distance > 0:
                        heuristic[i][j] = 1.0 / distance
        return heuristic

    def _select_next_city(self, current_city: int, unvisited: set,
                          pheromone: np.ndarray, heuristic: np.ndarray) -> int:
        """
             .

        P_ij = _ij^ * _ij^ / (_ik^ * _ik^)
```

```
        """
        probabilities = np.zeros(self.n_cities)

        for city in unvisited:
            tau = self.pheromone[current_city][city] ** self.alpha
            eta = heuristic[current_city][city] ** self.beta
            probabilities[city] = tau * eta

        #
        prob_sum = probabilities.sum()
        if prob_sum == 0:
            #    ,
            return np.random.choice(list(unvisited))

        probabilities = probabilities / prob_sum
        return np.random.choice(self.n_cities, p=probabilities)

    def _construct_tour(self, heuristic: np.ndarray) -> List[int]:
        """   ."""
        start_city = np.random.randint(0, self.n_cities)
        tour = [start_city]
        unvisited = set(range(self.n_cities)) - {start_city}

        while unvisited:
            next_city = self._select_next_city(tour[-1], unvisited,
                                               self.pheromone, heuristic)
            tour.append(next_city)
            unvisited.remove(next_city)

        return tour

    def _update_pheromone(self, tours: List[List[int]], distances: List[float]):
        """
          .
        _ij = (1 - ) * _ij + (_ij)
        """
        #
        self.pheromone *= (1 - self.rho)

        #
        for tour, distance in zip(tours, distances):
            delta_pheromone = self.Q / distance
            for i in range(len(tour)):
                from_city = tour[i]
                to_city = tour[(i + 1) % len(tour)]
                self.pheromone[from_city][to_city] += delta_pheromone
                self.pheromone[to_city][from_city] += delta_pheromone

    def solve(self) -> Tuple[List[int], float, Dict]:
        """
            .

        Returns:
```

```
            (best_tour, best_distance, metrics)
        """
        start_time = time.time()

        heuristic = self._calculate_heuristic()
        iterations = 0

        for iteration in range(self.n_iterations):
            #
            tours = []
            distances = []

            for _ in range(self.n_ants):
                tour = self._construct_tour(heuristic)
                distance = self.graph.calculate_tour_length(tour)
                tours.append(tour)
                distances.append(distance)

                #
                if distance < self.best_distance:
                    self.best_tour = tour.copy()
                    self.best_distance = distance

            #
            self._update_pheromone(tours, distances)

            #
            self.history.append(self.best_distance)
            iterations += 1

        elapsed_time = time.time() - start_time

        metrics = {
            'algorithm': 'Ant Colony Optimization (Basic)',
            'best_distance': self.best_distance,
            'iterations': iterations,
            'time': elapsed_time,
            'function_evaluations': iterations * self.n_ants,
            'n_ants': self.n_ants,
        }

        return self.best_tour, self.best_distance, metrics


class AntColonyOptimizationInitial(AntColonyOptimization):
    """
       _ " ".

    :
        .
          .
    """
```

```
    def __init__(self, graph, n_ants: int = 30, n_iterations: int = 100,
                 alpha: float = 1.0, beta: float = 2.0,
                 rho: float = 0.1, Q: float = 100):
        """
        Args:
            graph: Graph
            n_ants:
            n_iterations:
            alpha:
            beta:
            rho:
            Q:  ,
        """
        super().__init__(graph, n_ants, n_iterations, alpha, beta, rho, Q)
        self.initial_phase = True

    def _construct_tour_from_city(self, start_city: int, heuristic: np.ndarray) -> List[int]:
        """  ,    ."""
        tour = [start_city]
        unvisited = set(range(self.n_cities)) - {start_city}

        while unvisited:
            next_city = self._select_next_city(tour[-1], unvisited,
                                               self.pheromone, heuristic)
            tour.append(next_city)
            unvisited.remove(next_city)

        return tour

    def solve(self) -> Tuple[List[int], float, Dict]:
        """
              .



         .

        Returns:
            (best_tour, best_distance, metrics)
        """
        start_time = time.time()

        heuristic = self._calculate_heuristic()
        iterations = 0

        for iteration in range(self.n_iterations):
            tours = []
            distances = []

            if iteration == 0:
                #  :
                for ant_id in range(self.n_ants):
                    start_city = ant_id % self.n_cities
                    tour = self._construct_tour_from_city(start_city, heuristic)
```

```
                    distance = self.graph.calculate_tour_length(tour)
                    tours.append(tour)
                    distances.append(distance)

                    if distance < self.best_distance:
                        self.best_tour = tour.copy()
                        self.best_distance = distance
            else:
                #  :
                for _ in range(self.n_ants):
                    tour = self._construct_tour(heuristic)
                    distance = self.graph.calculate_tour_length(tour)
                    tours.append(tour)
                    distances.append(distance)

                    if distance < self.best_distance:
                        self.best_tour = tour.copy()
                        self.best_distance = distance

            #
            self._update_pheromone(tours, distances)

            #
            self.history.append(self.best_distance)
            iterations += 1

        elapsed_time = time.time() - start_time

        metrics = {
            'algorithm': 'Ant Colony Optimization (Initial Placement)',
            'best_distance': self.best_distance,
            'iterations': iterations,
            'time': elapsed_time,
            'function_evaluations': iterations * self.n_ants,
            'n_ants': self.n_ants,
        }

        return self.best_tour, self.best_distance, metrics
```

Листинг 5. Исходный код файла `src/ui.py`

```
import os
import math
import sys
import time
from dataclasses import dataclass

from PyQt6.QtCore import QPointF, Qt, QThread, pyqtSignal
from PyQt6.QtGui import QBrush, QColor, QFont, QPainter, QPen
from PyQt6.QtWidgets import (
    QApplication,
    QComboBox,
    QDoubleSpinBox,
```

```
    QFormLayout,
    QGridLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QPlainTextEdit,
    QSpinBox,
    QStackedWidget,
    QVBoxLayout,
    QWidget,
)

from .aco import AntColonyOptimization, AntColonyOptimizationInitial
from .annealing import BoltzmannAnnealing, SimulatedAnnealing
from .graph_utils import Graph


@dataclass
class RunResult:
    algorithm: str
    graph_name: str
    n_cities: int
    layout_points: list[tuple[float, float]]
    best_distance: float
    iterations: int
    elapsed: float
    best_tour: list[int]
    history: list[float]


class RouteVisualizationWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._n_cities = 0
        self._layout_points: list[tuple[float, float]] = []
        self._best_tour: list[int] = []
        self._graph_name = ""
        self._best_distance = 0.0
        self.setMinimumHeight(300)
        self.setStyleSheet("border: 1px solid #c7c7c7; background: white;")

    def clear(self):
        self._n_cities = 0
        self._layout_points = []
        self._best_tour = []
        self._graph_name = ""
        self._best_distance = 0.0
        self.update()

    def set_route(self, graph_name: str, n_cities: int, layout_points: list[tuple[float, floa
```

```
        self._graph_name = graph_name
        self._n_cities = n_cities
        self._layout_points = list(layout_points)
        self._best_tour = list(best_tour)
        self._best_distance = best_distance
        self.update()

    def _build_positions(self, rect):
        if self._n_cities <= 0 or not self._layout_points:
            return []

        xs = [point[0] for point in self._layout_points]
        ys = [point[1] for point in self._layout_points]
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)
        span_x = max(max_x - min_x, 1e-9)
        span_y = max(max_y - min_y, 1e-9)

        available_width = max(rect.width(), 1)
        available_height = max(rect.height(), 1)
        scale = min(available_width / span_x, available_height / span_y) * 0.92

        scaled_width = span_x * scale
        scaled_height = span_y * scale
        offset_x = rect.left() + (available_width - scaled_width) / 2.0
        offset_y = rect.top() + (available_height - scaled_height) / 2.0

        positions = []
        for x_value, y_value in self._layout_points:
            x = offset_x + (x_value - min_x) * scale
            y = offset_y + (y_value - min_y) * scale
            positions.append(QPointF(x, y))

        return positions

    def paintEvent(self, event):
        super().paintEvent(event)

        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)
        rect = self.rect().adjusted(24, 56, -24, -24)

        painter.setPen(QPen(QColor("#1f2937"), 1))
        painter.setFont(QFont("Helvetica Neue", 11))
        title = self._graph_name if self._graph_name else "  "
        header = f"{title}  |  : {self._best_distance:.2f}" if self._best_tour else title
        painter.drawText(16, 24, header)

        if self._n_cities <= 0:
            painter.setPen(QColor("#6b7280"))
            painter.drawText(rect, Qt.AlignmentFlag.AlignCenter, " ,   ")
            return
```

```
        positions = self._build_positions(rect)
        if not positions:
            return

        if self._best_tour and len(self._best_tour) == self._n_cities:
            painter.setPen(QPen(QColor("#2563eb"), 2))
            for index, current_city in enumerate(self._best_tour):
                next_city = self._best_tour[(index + 1) % len(self._best_tour)]
                painter.drawLine(positions[current_city], positions[next_city])

        painter.setFont(QFont("Helvetica Neue", 9))
        label_needed = self._n_cities <= 120

        for index, point in enumerate(positions):
            painter.setPen(Qt.PenStyle.NoPen)
            painter.setBrush(QBrush(QColor("#111827")))
            painter.drawEllipse(point, 4.5, 4.5)

            painter.setPen(QColor("#111827"))
            painter.setBrush(Qt.BrushStyle.NoBrush)
            if label_needed:
                painter.drawText(QPointF(point.x() + 6, point.y() - 6), str(index + 1))


class HistoryPlotWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._history: list[float] = []
        self._graph_name = ""
        self.setMinimumHeight(240)
        self.setStyleSheet("border: 1px solid #c7c7c7; background: white;")

    def clear(self):
        self._history = []
        self._graph_name = ""
        self.update()

    def set_history(self, graph_name: str, history: list[float]):
        self._graph_name = graph_name
        self._history = list(history)
        self.update()

    def paintEvent(self, event):
        super().paintEvent(event)

        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)

        painter.setPen(QPen(QColor("#1f2937"), 1))
        painter.setFont(QFont("Helvetica Neue", 11))
        title = self._graph_name if self._graph_name else "  "
        painter.drawText(16, 24, "  : " + title)
```

```
        rect = self.rect().adjusted(48, 44, -20, -28)
        painter.setPen(QPen(QColor("#d1d5db"), 1))
        painter.drawRect(rect)

        if not self._history:
            painter.setPen(QColor("#6b7280"))
            painter.drawText(rect, Qt.AlignmentFlag.AlignCenter, " ,   ")
            return

        if len(self._history) == 1:
            painter.setPen(QPen(QColor("#2563eb"), 3))
            center_point = QPointF(rect.center())
            painter.drawEllipse(center_point, 3.5, 3.5)
            return

        min_value = min(self._history)
        max_value = max(self._history)
        if max_value - min_value < 1e-9:
            max_value = min_value + 1.0

        left = rect.left()
        right = rect.right()
        top = rect.top()
        bottom = rect.bottom()
        width = max(rect.width(), 1)
        height = max(rect.height(), 1)

        def map_point(index: int, value: float) -> QPointF:
            x_ratio = index / (len(self._history) - 1)
            y_ratio = (value - min_value) / (max_value - min_value)
            x = left + x_ratio * width
            y = bottom - y_ratio * height
            return QPointF(x, y)

        painter.setPen(QPen(QColor("#9ca3af"), 1))
        painter.drawLine(left, bottom, right, bottom)
        painter.drawLine(left, top, left, bottom)

        painter.setPen(QPen(QColor("#2563eb"), 2))
        previous_point = map_point(0, self._history[0])
        for index, value in enumerate(self._history[1:], start=1):
            current_point = map_point(index, value)
            painter.drawLine(previous_point, current_point)
            previous_point = current_point

        painter.setBrush(QBrush(QColor("#2563eb")))
        painter.setPen(Qt.PenStyle.NoPen)
        for index, value in enumerate(self._history):
            point = map_point(index, value)
            painter.drawEllipse(point, 2.5, 2.5)

        painter.setPen(QColor("#374151"))
        painter.setFont(QFont("Helvetica Neue", 9))
```

```
        painter.drawText(rect.left(), rect.top() - 6, f"min: {min_value:.2f}")
        painter.drawText(rect.right() - 140, rect.top() - 6, f"max: {max_value:.2f}")


class AlgorithmWorker(QThread):
    finished = pyqtSignal(object)
    failed = pyqtSignal(str)

    def __init__(self, graph_name: str, algorithm_name: str, params: dict):
        super().__init__()
        self.graph_name = graph_name
        self.algorithm_name = algorithm_name
        self.params = params

    def _load_graph(self) -> Graph:
        if self.graph_name == "small":
            return Graph.create_small_graph()
        if self.graph_name == "berlin52":
            return Graph.load_from_stp("data/berlin52.stp")
        return Graph.load_from_stp("data/world666.stp")

    def run(self):
        try:
            graph = self._load_graph()
            start_time = time.time()

            if self.algorithm_name == "sa_basic":
                algo = SimulatedAnnealing(
                    graph,
                    initial_temp=self.params["initial_temp"],
                    cooling_rate=self.params["cooling_rate"],
                    max_iterations=self.params["max_iterations"],
                )
            elif self.algorithm_name == "sa_boltzmann":
                algo = BoltzmannAnnealing(
                    graph,
                    initial_temp=self.params["initial_temp"],
                    max_iterations=self.params["max_iterations"],
                )
            elif self.algorithm_name == "aco_basic":
                algo = AntColonyOptimization(
                    graph,
                    n_ants=self.params["n_ants"],
                    n_iterations=self.params["n_iterations"],
                    alpha=self.params["alpha"],
                    beta=self.params["beta"],
                    rho=self.params["rho"],
                    Q=self.params["Q"],
                )
            else:
                algo = AntColonyOptimizationInitial(
                    graph,
                    n_ants=self.params["n_ants"],
```

```
                    n_iterations=self.params["n_iterations"],
                    alpha=self.params["alpha"],
                    beta=self.params["beta"],
                    rho=self.params["rho"],
                    Q=self.params["Q"],
                )

            best_tour, best_distance, metrics = algo.solve()
            elapsed = time.time() - start_time

            result = RunResult(
                algorithm=metrics["algorithm"],
                graph_name=self.graph_name,
                n_cities=graph.n_cities,
                layout_points=[(float(x), float(y)) for x, y in graph.get_visualization_layou
                best_distance=float(best_distance),
                iterations=int(metrics["iterations"]),
                elapsed=float(elapsed),
                best_tour=list(best_tour),
                history=[float(value) for value in getattr(algo, "history", [])],
            )
            self.finished.emit(result)
        except Exception as exc:
            self.failed.emit(str(exc))


class AlgorithmPage(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.graph_combo = QComboBox()
        self.graph_combo.addItems(["small", "berlin52", "world666"])

        self.algorithm_combo = QComboBox()
        self.algorithm_combo.addItems(["sa_basic", "sa_boltzmann", "aco_basic", "aco_initial'

        self.run_button = QPushButton("")
        self.run_button.clicked.connect(self.run_algorithm)

        self.log = QPlainTextEdit()
        self.log.setReadOnly(True)

        self.route_view = RouteVisualizationWidget()
        self.history_view = HistoryPlotWidget()

        self.param_stack = QStackedWidget()
        self.param_stack.addWidget(self._build_sa_page())
        self.param_stack.addWidget(self._build_sa_boltzmann_page())
        self.param_stack.addWidget(self._build_aco_page())
        self.param_stack.addWidget(self._build_aco_page(initial=True))

        self.algorithm_combo.currentIndexChanged.connect(self.param_stack.setCurrentIndex)
```

```
        top_box = QGroupBox(" ")
        top_layout = QGridLayout(top_box)
        top_layout.addWidget(QLabel(""), 0, 0)
        top_layout.addWidget(self.graph_combo, 0, 1)
        top_layout.addWidget(QLabel(""), 1, 0)
        top_layout.addWidget(self.algorithm_combo, 1, 1)
        top_layout.addWidget(self.run_button, 2, 0, 1, 2)

        main_layout = QHBoxLayout(self)

        left_column = QVBoxLayout()
        left_column.addWidget(top_box)
        left_column.addWidget(self.param_stack)
        left_column.addWidget(QLabel(""))
        left_column.addWidget(self.log)

        right_column = QVBoxLayout()
        right_column.addWidget(self.route_view)
        right_column.addWidget(self.history_view)

        main_layout.addLayout(left_column, 2)
        main_layout.addLayout(right_column, 1)

    def _build_sa_page(self):
        page = QWidget()
        form = QFormLayout(page)

        self.sa_initial_temp = QDoubleSpinBox()
        self.sa_initial_temp.setRange(1.0, 100000.0)
        self.sa_initial_temp.setValue(1000.0)

        self.sa_cooling_rate = QDoubleSpinBox()
        self.sa_cooling_rate.setRange(0.01, 0.9999)
        self.sa_cooling_rate.setSingleStep(0.01)
        self.sa_cooling_rate.setDecimals(4)
        self.sa_cooling_rate.setValue(0.95)

        self.sa_max_iterations = QSpinBox()
        self.sa_max_iterations.setRange(1, 200000)
        self.sa_max_iterations.setValue(5000)

        form.addRow(" ", self.sa_initial_temp)
        form.addRow(" ", self.sa_cooling_rate)
        form.addRow(". ", self.sa_max_iterations)
        return page

    def _build_sa_boltzmann_page(self):
        page = QWidget()
        form = QFormLayout(page)

        self.b_initial_temp = QDoubleSpinBox()
        self.b_initial_temp.setRange(1.0, 100000.0)
        self.b_initial_temp.setValue(1000.0)
```

```
        self.b_max_iterations = QSpinBox()
        self.b_max_iterations.setRange(1, 200000)
        self.b_max_iterations.setValue(5000)

        form.addRow(" ", self.b_initial_temp)
        form.addRow(". ", self.b_max_iterations)
        return page

    def _build_aco_page(self, initial=False):
        page = QWidget()
        form = QFormLayout(page)

        ants = QSpinBox()
        ants.setRange(1, 1000)
        ants.setValue(20)

        iterations = QSpinBox()
        iterations.setRange(1, 10000)
        iterations.setValue(50)

        alpha = QDoubleSpinBox()
        alpha.setRange(0.1, 10.0)
        alpha.setSingleStep(0.1)
        alpha.setValue(1.0)

        beta = QDoubleSpinBox()
        beta.setRange(0.1, 10.0)
        beta.setSingleStep(0.1)
        beta.setValue(2.0)

        rho = QDoubleSpinBox()
        rho.setRange(0.01, 0.99)
        rho.setSingleStep(0.01)
        rho.setDecimals(2)
        rho.setValue(0.1)

        q_value = QDoubleSpinBox()
        q_value.setRange(1.0, 10000.0)
        q_value.setValue(100.0)

        form.addRow(" ", ants)
        form.addRow(" ", iterations)
        form.addRow("alpha", alpha)
        form.addRow("beta", beta)
        form.addRow("rho", rho)
        form.addRow("Q", q_value)

        if initial:
            self.aco_initial_ants = ants
            self.aco_initial_iterations = iterations
            self.aco_initial_alpha = alpha
            self.aco_initial_beta = beta
```

```
            self.aco_initial_rho = rho
            self.aco_initial_q = q_value
        else:
            self.aco_basic_ants = ants
            self.aco_basic_iterations = iterations
            self.aco_basic_alpha = alpha
            self.aco_basic_beta = beta
            self.aco_basic_rho = rho
            self.aco_basic_q = q_value

        return page

    def _get_params(self):
        algo = self.algorithm_combo.currentText()
        if algo == "sa_basic":
            return {
                "initial_temp": self.sa_initial_temp.value(),
                "cooling_rate": self.sa_cooling_rate.value(),
                "max_iterations": self.sa_max_iterations.value(),
            }
        if algo == "sa_boltzmann":
            return {
                "initial_temp": self.b_initial_temp.value(),
                "max_iterations": self.b_max_iterations.value(),
            }
        if algo == "aco_basic":
            return {
                "n_ants": self.aco_basic_ants.value(),
                "n_iterations": self.aco_basic_iterations.value(),
                "alpha": self.aco_basic_alpha.value(),
                "beta": self.aco_basic_beta.value(),
                "rho": self.aco_basic_rho.value(),
                "Q": self.aco_basic_q.value(),
            }
        return {
            "n_ants": self.aco_initial_ants.value(),
            "n_iterations": self.aco_initial_iterations.value(),
            "alpha": self.aco_initial_alpha.value(),
            "beta": self.aco_initial_beta.value(),
            "rho": self.aco_initial_rho.value(),
            "Q": self.aco_initial_q.value(),
        }

    def run_algorithm(self):
        self.run_button.setEnabled(False)
        self.route_view.clear()
        self.history_view.clear()
        self.log.appendPlainText("...\n")

        self.worker = AlgorithmWorker(
            self.graph_combo.currentText(),
            self.algorithm_combo.currentText(),
            self._get_params(),
```

```
        )
        self.worker.finished.connect(self._on_finished)
        self.worker.failed.connect(self._on_failed)
        self.worker.start()

    def _on_finished(self, result):
        self.run_button.setEnabled(True)
        self.route_view.set_route(result.graph_name, result.n_cities, result.layout_points, r
        self.history_view.set_history(result.graph_name, result.history)
        self.log.appendPlainText(
            f": {result.algorithm}\n"
            f": {result.graph_name}\n"
            f" : {result.best_distance}\n"
            f": {result.iterations}\n"
            f": {result.elapsed:.4f} \n"
            f": {result.best_tour}\n"
            "----------------------------------------\n"
        )

    def _on_failed(self, message):
        self.run_button.setEnabled(True)
        self.route_view.clear()
        self.history_view.clear()
        QMessageBox.critical(self, "", message)


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("TSP: SA  ACO")
        self.resize(1400, 800)
        self.setCentralWidget(AlgorithmPage(self))


def run_app():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```